Project- High Level Design
On
IaC Provisioning for Real Estate System

Course Name: DevOps

Medicaps University

Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr no	Student Name	Enrolment Number
1	Sandesh Vani	EN22CS301862
2	Shivam Choudhary	EN22CS301903
3	Shreya Gupta	EN22CS301927
4	Shrishti Chouksey	EN22CS301935
5	Shyam Narayan Pandey	EN22CS301949
6	Siddhi Suraniya	EN22CS301956

Group Name: 12 D8

Project Number: DO-47

Industry Mentor Name:

University Mentor Name: Prof. Avnesh Kumar Joshi

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document
   1.2. Intended Audience
   1.3. System Overview

2. System Design
   2.1. Application Design
   2.2. Process Flow
   2.3. Information Flow
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue

3. Data Design
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

INTRODUCTION

- ## Scope of the document

  The scope of this document is to define the high-level design and implementation approach for automating infrastructure provisioning in a Real Estate System using Infrastructure as Code (IaC) practices. It outlines the overall architecture, tools, workflows, and processes required to enable a fully automated, scalable, and consistent environment setup. The document focuses on leveraging modern DevOps tools such as Git for source code management, Docker for containerization, Jenkins and GitHub Actions for continuous integration and deployment, and Kubernetes for container orchestration. Additionally, it includes the use of IaC tools such as Terraform or Ansible to provision and manage infrastructure resources either locally using Docker environments or on cloud platforms like Amazon Web Services. The scope also covers automated installation of required dependencies, environment configuration, and deployment pipelines that ensure consistency across development, testing, and production environments. It aims to eliminate manual intervention, reduce configuration errors, and enable repeatable deployments. Furthermore, the document highlights integration between various components of the system, including version control, build pipelines, containerization, and orchestration layers, ensuring seamless end-to-end automation. Security, scalability, and maintainability considerations are also included within the scope to support future enhancements and production readiness. Overall, this document serves as a comprehensive guide for designing and implementing an efficient, reliable, and automated infrastructure provisioning system tailored for a Real Estate application using modern DevOps and IaC methodologies.

- ## Intended Audience

  The intended audience for this document includes all stakeholders involved in the design, development, deployment, and maintenance of the Real Estate System infrastructure. This primarily includes DevOps engineers, system administrators, cloud engineers, and software developers who are responsible for implementing and managing the Infrastructure as Code (IaC) workflows. Professionals working with tools such as Git, Docker, Jenkins, GitHub Actions, and Kubernetes will find this document particularly useful, as it provides insights into how these technologies are integrated within the system. Additionally, cloud architects and infrastructure engineers working with platforms like Amazon Web Services will benefit from understanding the provisioning strategies using IaC tools such as Terraform and Ansible. The document is also relevant for project managers and technical leads who need a high-level understanding of the system architecture, deployment workflows, and automation strategies to make informed decisions. Furthermore, quality assurance (QA) engineers and testers can use this document to understand the environment setup and deployment pipeline for effective testing. Overall, the document is designed for both technical and semi-technical audiences who are involved in ensuring the successful delivery, scalability, and maintenance of the Real Estate system through modern DevOps and IaC practices.

- **System Overview**

  The Real Estate System is designed as a modern, scalable, and automated platform that leverages DevOps practices and Infrastructure as Code (IaC) to ensure efficient infrastructure provisioning and application deployment. The system architecture integrates multiple tools and technologies to enable seamless development, continuous integration, continuous deployment, and container orchestration. At its core, the system uses Git as the central repository for managing source code, configuration files, and infrastructure scripts, ensuring version control and collaboration among team members. Application components are containerized using Docker, which provides consistency across different environments by packaging the application and its dependencies into lightweight, portable containers.

  For automation, the system incorporates CI/CD pipelines using Jenkins and GitHub Actions, enabling automated build, test, and deployment processes. These pipelines are triggered by code changes, ensuring rapid and reliable delivery of application updates. Infrastructure provisioning is handled through IaC tools such as Terraform or Ansible, which automate the creation and configuration of resources either on local environments or cloud platforms like Amazon Web Services. This eliminates manual setup and ensures repeatability.

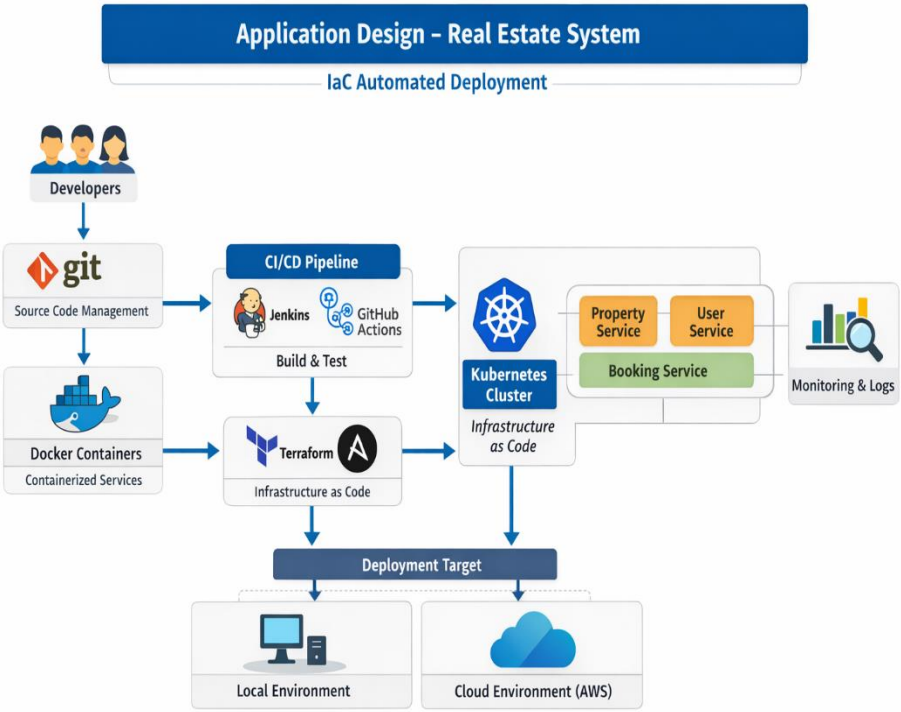
  The deployed containers are managed and orchestrated using Kubernetes, which provides features like auto-scaling, load balancing, and high availability. The system supports both local development environments and cloud-based deployments, making it flexible and scalable according to business needs. Overall, the Real Estate System ensures a robust, automated, and efficient workflow from code development to production deployment, minimizing errors, improving reliability, and supporting continuous delivery.

SYSTEM DESIGN

- **Application Design**

  - The Real Estate System follows a **modular and microservices-based architecture**, where each component (such as user management, property listings, and booking services) is developed and deployed independently for better scalability and maintainability.
  - The application source code is managed using Git, enabling version tracking, collaboration, and seamless integration with CI/CD pipelines.
  - Each service of the application is containerized using Docker, ensuring consistency across development, testing, and production environments.
  - The application is designed to be **cloud-agnostic**, allowing deployment on local environments or cloud platforms such as Amazon Web Services.

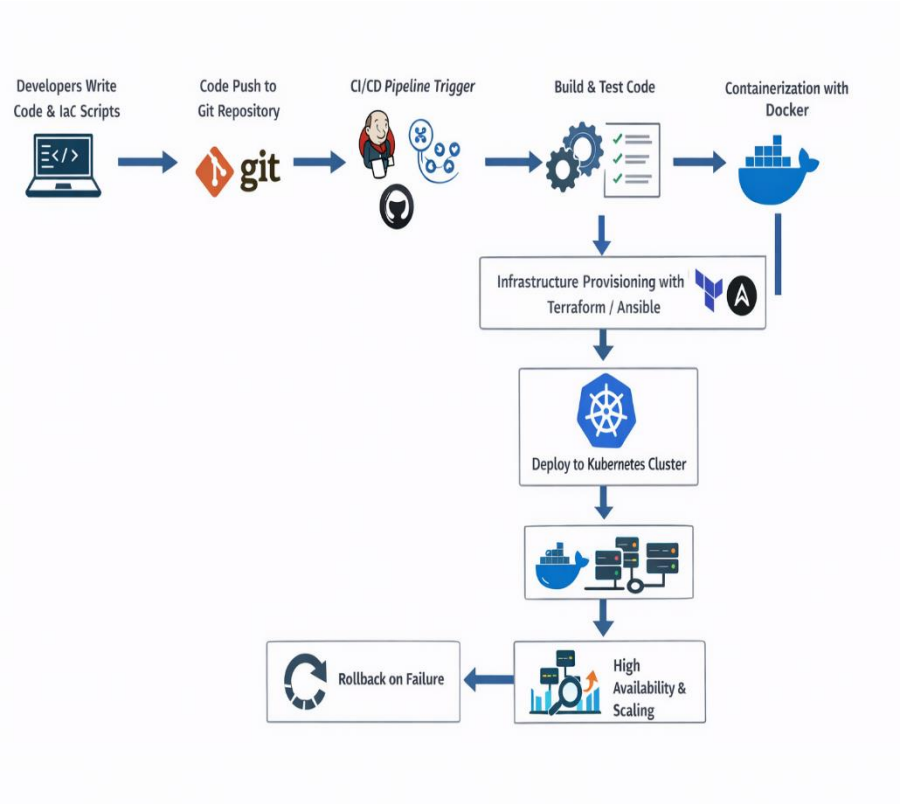
- CI/CD pipelines are implemented using Jenkins and GitHub Actions to automate build, test, and deployment processes.
- The application supports **RESTful APIs** for communication between frontend and backend services, ensuring smooth data exchange and integration.
- Environment configurations and secrets are managed securely using environment variables and external configuration management practices.
- The application is deployed and managed using Kubernetes, which ensures high availability, load balancing, and auto-scaling of services.
- Logging and monitoring mechanisms are integrated to track system performance, detect failures, and ensure system reliability.
- The design ensures **high scalability, fault tolerance, and maintainability**, making the system capable of handling increasing user demands and future enhancements efficiently.



- **Process Flow**

  - The process begins with developers writing application code and infrastructure scripts, which are then pushed to the central repository managed using Git.
  - Once the code is committed, an automated trigger initiates the CI/CD pipeline configured in Jenkins or GitHub Actions.
  - The pipeline starts with the **build phase**, where the application code is compiled and dependencies are installed.

- In the next step, automated **testing** is performed to ensure code quality, functionality, and stability before deployment.
- After successful testing, the application is packaged into containers using Docker to ensure environment consistency.
- Simultaneously, infrastructure provisioning is executed using IaC tools such as Terraform or Ansible to set up required resources.
- The containerized application is then deployed to a cluster managed by Kubernetes for orchestration and scaling.
- Kubernetes handles **service discovery, load balancing, and auto-scaling** to ensure high availability of the application.
- Post-deployment, **monitoring and logging mechanisms** track system performance and detect any issues in real time.
- If any failure occurs, the pipeline supports **rollback mechanisms** to revert to a stable version, ensuring reliability and minimal downtime.



- **Information Flow**

  - The information flow begins when users interact with the Real Estate application through the frontend interface, submitting requests such as property searches, registrations, or bookings.
  - These requests are routed to backend services via **RESTful APIs**, ensuring structured and secure communication between client and server components.
  - The application source code and configuration updates are managed through Git, which acts as the central repository for all changes.

- Once code is updated, CI/CD pipelines in Jenkins or GitHub Actions process the changes and propagate them through build, test, and deployment stages.
- During the build phase, application data and dependencies are packaged into containers using Docker, ensuring consistent runtime environments.
- Infrastructure-related information, such as resource configurations and environment setup, flows through IaC tools like Terraform or Ansible.
- The deployed services communicate internally within the cluster managed by Kubernetes, enabling efficient service-to-service data exchange.
- Data generated by user interactions and system processes is stored and retrieved from databases, ensuring persistence and consistency.
- Monitoring and logging tools capture system metrics, logs, and events, providing real-time visibility into application performance and issues.
- Feedback from monitoring systems flows back to developers and operations teams, enabling continuous improvement, faster debugging, and system optimization.

- **Components Design**

  - The system is divided into **independent components (microservices)** such as User Service, Property Service, Booking Service, and Authentication Service, ensuring modularity and ease of maintenance.
  - The **Frontend Component** provides the user interface for buyers, sellers, and agents to interact with the system, sending requests to backend services via APIs.
  - The **Backend Component** handles business logic, request processing, and communication between different services, ensuring proper data flow and system functionality.
  - The **API Gateway Component** acts as a single entry point for all client requests, managing routing, authentication, and request validation.
  - The **Version Control Component** uses Git to manage source code, track changes, and enable collaboration among development teams.
  - The **CI/CD Component** is implemented using Jenkins and GitHub Actions to automate build, testing, and deployment processes.
  - The **Containerization Component** uses Docker to package applications and dependencies into portable containers.
  - The **Infrastructure Provisioning Component** leverages terraform or ansible to automate environment setup and configuration.
  - The **Orchestration Component** is managed by kubernetes, which handles deployment, scaling, and load balancing of containers.
  - The **Monitoring and Logging Component** tracks system health, performance metrics, and logs, enabling quick issue detection and improving system reliability.

- **Key Design Considerations**

The key design considerations for the Real Estate System focus on ensuring scalability, reliability, security, and maintainability while implementing Infrastructure as Code (IaC) and DevOps practices. One of the primary considerations is scalability, where the system is designed using containerization with Docker and orchestration through Kubernetes to handle increasing user loads efficiently. Reliability and fault tolerance are achieved through automated CI/CD pipelines using Jenkins and GitHub Actions, enabling continuous testing, deployment, and quick rollback in case of failures. Another important aspect is consistency, which is maintained by using IaC tools such as Terraform or Ansible to automate infrastructure provisioning and eliminate manual errors. Security is also a critical consideration, including secure handling of credentials, role-based access control, and protected communication between services. Version control using Git ensures proper tracking of code and configuration changes. Additionally, the system is designed to be flexible and cloud-compatible, allowing deployment on platforms like Amazon Web Services. Monitoring and logging mechanisms are incorporated to maintain system health and performance. Overall, these considerations ensure a robust, efficient, and production-ready system.

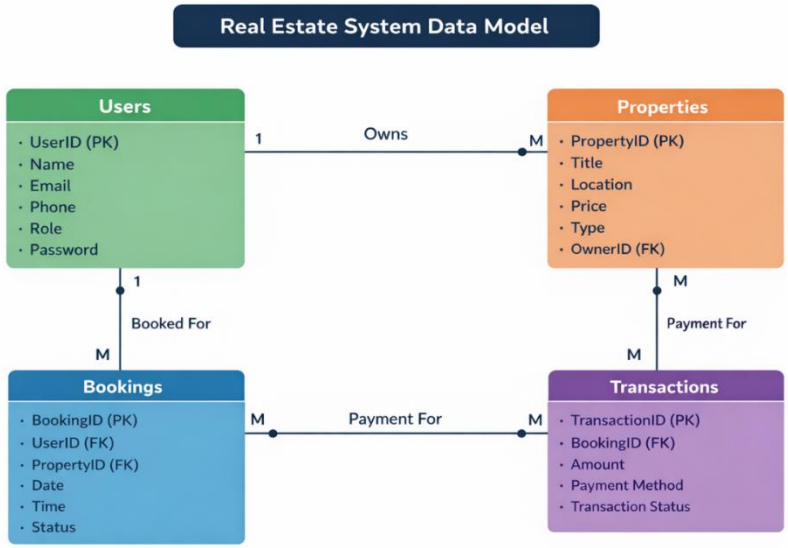
- **API Catalogue**

The API Catalogue for the Real Estate System defines a structured set of RESTful APIs that enable communication between the frontend, backend services, and external systems. These APIs are designed to support core functionalities such as user management, property listing, search operations, booking processes, and authentication. The User API handles operations like user registration, login, profile management, and role-based access control, ensuring secure interaction within the system. The Property API allows users to add, update, delete, and view property listings, along with advanced filtering and search capabilities based on location, price, and property type. The Booking API manages scheduling of property visits, booking confirmations, and cancellations, ensuring smooth coordination between buyers and sellers. An Authentication API is implemented to manage secure login sessions using token-based authentication methods such as JWT, ensuring data protection and secure access. All APIs are version-controlled and documented, with changes managed through Git to maintain consistency and traceability. These APIs are integrated into automated CI/CD pipelines using Jenkins and GitHub Actions, ensuring that updates are tested and deployed efficiently. The APIs are containerized using Docker and deployed on Kubernetes, enabling scalability and high availability. Additionally, API gateways and routing mechanisms ensure proper request handling, load balancing, and security enforcement. Overall, the API Catalogue provides a well-defined interface layer that ensures seamless communication, modularity, and extensibility of the Real Estate System while supporting modern DevOps and IaC practices.

DATA DESIGN

- **Data Model**

  - The data model is designed to represent core entities of the Real Estate System such as **Users, Properties, Bookings, and Transactions**, ensuring structured data management.
  - The **User Entity** stores details like user ID, name, email, contact information, role (buyer, seller, admin), and authentication credentials.
  - The **Property Entity** contains information such as property ID, title, description, location, price, type, availability status, and associated owner (user).
  - The **Booking Entity** manages scheduling details including booking ID, user ID, property ID, date, time, and booking status (confirmed, pending, cancelled).
  - The **Transaction Entity** records payment-related details such as transaction ID, amount, payment method, and transaction status.
  - Relationships are established between entities, such as **one-to-many** (one user can list multiple properties) and **many-to-one** (multiple bookings for a single property).
  - The data model ensures **data normalization** to reduce redundancy and maintain data integrity across the system.
  - Unique identifiers (primary keys) and references (foreign keys) are used to maintain relationships and ensure consistency.
  - The system supports scalable storage solutions, allowing deployment on cloud platforms like Amazon Web Services for better performance and availability.
  - Data access is secured through authentication and authorization mechanisms, ensuring that only authorized users can access or modify sensitive information.



- **Data Access Mechanism**

  - The system follows a **layered architecture**, where data access is handled through a dedicated Data Access Layer (DAL) to separate business logic from database operations.
  - All data interactions are performed via **RESTful APIs**, ensuring controlled and secure communication between frontend and backend services.
  - The backend services use **ORM (Object Relational Mapping)** frameworks to interact with the database, reducing complexity and improving maintainability.
  - Database queries are optimized using indexing and efficient query design to ensure **fast data retrieval and performance**.
  - Authentication and authorization mechanisms ensure that only **authorized users** can access or modify data, maintaining data security.
  - Sensitive data such as passwords and credentials are stored using **encryption and hashing techniques** to prevent unauthorized access.
  - The system supports **transaction management** to maintain data consistency, especially during booking and payment operations.
  - Data is accessed and managed in containerized environments using Docker, ensuring consistency across environments.
  - Infrastructure and database configurations are provisioned using IaC tools like Terraform or Ansible for automated setup.
  - The deployed services running on Kubernetes ensure high availability and scalable access to data across multiple instances.

- **Data Retention Policies**

- The system defines clear **data retention periods** for different data types such as user data, property listings, bookings, and transaction records based on business and compliance requirements.
- Critical data like financial transactions and booking history is retained for a **longer duration** to support auditing, reporting, and legal obligations.
- Temporary or less critical data, such as session logs and cache data, is stored for a **short duration** and automatically deleted after expiry.
- User data is retained only as long as necessary, and users are provided options for **account deletion or data removal** as per privacy policies.
- Regular **data archiving mechanisms** are implemented to move older data to cost-effective storage while keeping it accessible if required.
- Automated scripts and workflows ensure periodic **data cleanup and deletion**, reducing storage overhead and improving system performance.
- Data backups are maintained at regular intervals to ensure **data recovery in case of failures**, with retention policies applied to backup storage as well.
- The entire data lifecycle is managed securely, with infrastructure provisioning and policies enforced using IaC tools like Terraform or Ansible to maintain consistency and compliance.

- ## Data Migration

Data migration in the Real Estate System is a critical process that ensures the seamless transfer of data from legacy systems or existing databases to the newly designed infrastructure powered by Infrastructure as Code (IaC). This process involves careful planning, validation, and execution to maintain data integrity, consistency, and minimal downtime. The migration strategy typically includes stages such as data extraction, transformation, and loading (ETL), where existing data is cleaned, formatted, and mapped to the new data model. Tools and scripts are version-controlled using Git to maintain traceability and collaboration among teams. Automated pipelines configured in Jenkins or GitHub Actions are used to execute migration workflows, ensuring repeatability and reducing manual intervention.

During migration, data is securely transferred and validated at each stage to prevent data loss or corruption. Backup mechanisms are implemented before initiating migration to ensure recovery in case of failures. The use of containerization through Docker allows migration tools and scripts to run

consistently across different environments. Infrastructure provisioning using Terraform or Ansible ensures that the target environment is correctly set up before data is loaded. Once the data is migrated, it is deployed and managed within clusters orchestrated by Kubernetes, enabling scalability and high availability. Post-migration validation and testing are conducted to verify data accuracy and system functionality. Overall, the data migration approach ensures a smooth transition to the new system with minimal disruption, high reliability, and adherence to best practices in DevOps and IaC.

INTERFACES

The Interfaces in the Real Estate System define how different components of the application interact with each other as well as with external systems, ensuring smooth communication and data exchange across the entire architecture. The system primarily uses **RESTful APIs** as the main interface between the frontend and backend services, enabling users to perform operations such as property search, user registration, and booking management. These APIs are designed to be scalable, secure, and easy to integrate with various client applications. Internal service-to-service communication is handled through well-defined interfaces within a microservices architecture, ensuring loose coupling and independent deployment of components. The API definitions and interface contracts are maintained and version-controlled using Git, ensuring consistency and traceability of changes.

For automation and deployment, the system integrates with CI/CD tools such as Jenkins and GitHub Actions, which act as interfaces between code repositories and deployment environments, enabling seamless build and release workflows. Container interfaces are managed using Docker, allowing applications to run consistently across environments. At the infrastructure level, IaC tools like Terraform and Ansible provide interfaces for defining and managing infrastructure resources programmatically. The deployment and orchestration interface is handled by Kubernetes, which manages communication between services, load balancing, and scaling operations. Additionally, external interfaces may include integration with cloud platforms such as Amazon Web Services for hosting and storage services. Overall, these interfaces ensure interoperability, flexibility, and efficient communication across the Real Estate System.

STATE AND SESSION MANAGEMENT

- The system manages **state in a stateless manner** at the application level, ensuring scalability and easy distribution of services across multiple instances.
- **Session management** is implemented using token-based mechanisms such as JWT (JSON Web Tokens), allowing secure and efficient user authentication without storing session data on the server.
- User session data (like login status and permissions) is maintained on the client side and validated on each request through secure tokens.
- Backend services are designed to remain **stateless**, enabling seamless scaling using Kubernetes without dependency on a single server.
- Temporary session-related data, if required, can be stored in **in-memory data stores or caches** to improve performance and reduce database load.
- Session expiration and timeout mechanisms are implemented to ensure **security and automatic logout** after inactivity.

CACHING

Caching in the Real Estate System is implemented to enhance application performance, reduce latency, and minimize the load on backend services and databases. It plays a critical role in improving user experience by storing frequently accessed data such as property listings, search results, and user session information in temporary storage, allowing faster retrieval without repeated database queries. The system uses a layered caching approach, where caching can be

applied at multiple levels including client-side, application-level, and distributed caching mechanisms.

At the application level, caching is integrated within containerized services running on Docker, ensuring consistent behaviour across different environments. In a distributed architecture, caching works efficiently with Kubernetes, where multiple instances of services can access shared cached data, improving scalability and reducing redundant processing. Cache invalidation strategies are carefully designed to ensure that outdated or stale data is refreshed when updates occur, maintaining data consistency and accuracy.

The system also supports time-based expiration (TTL - Time to Live) policies, where cached data is automatically cleared after a specific duration to ensure up-to-date information. Secure handling of cached data is considered, avoiding storage of sensitive information such as passwords or confidential user details. Additionally, caching configurations and policies are managed through automated deployment pipelines using Jenkins and GitHub Actions, ensuring consistency across environments. Infrastructure provisioning tools like Terraform or Ansible help in setting up and managing caching layers efficiently. Overall, caching significantly improves system performance, scalability, and responsiveness in the Real Estate System.

NON-FUNCTIONAL REQUIREMENTS

- ## Security Aspects

  Security is a critical non-functional requirement for the Real Estate System, ensuring the protection of sensitive user data, financial transactions, and system resources from unauthorized access and potential threats. The system implements strong **authentication and authorization mechanisms**, including token-based authentication such as JWT, to verify user identity and enforce role-based access control (RBAC). All communication between client and server is secured using HTTPS protocols, ensuring data encryption during transmission and preventing interception or man-in-the-middle attacks.

  Sensitive information such as passwords and personal data is stored using secure hashing and encryption techniques, minimizing the risk of data breaches. The system also integrates secure coding practices and vulnerability scanning within CI/CD pipelines using tools like Jenkins and GitHub Actions to detect and fix security issues early in the development lifecycle. Container security is ensured through proper configuration and image scanning in Docker environments.

  At the infrastructure level, access controls, network security groups, and firewall configurations are managed using Infrastructure as Code tools such as Terraform or Ansible, ensuring consistent and secure environment setup. The use of Kubernetes further enhances security through namespace isolation, secrets management, and controlled access to services.

  Regular monitoring, logging, and auditing mechanisms are implemented to detect suspicious activities and respond to potential threats in real time. Additionally, session

management practices such as token expiration and secure storage prevent unauthorized reuse of credentials. Overall, the system adopts a multi-layered security approach, combining application-level, infrastructure-level, and operational security measures to ensure data integrity, confidentiality, and availability.

- **Performance Aspects**

Performance is a key non-functional requirement for the Real Estate System, ensuring that the application delivers fast response times, high throughput, and a seamless user experience even under heavy load conditions. The system is designed using a scalable architecture where containerized services deployed via Docker are orchestrated using Kubernetes, enabling horizontal scaling based on demand. This allows the system to efficiently handle increasing numbers of users and transactions without degradation in performance.

To further optimize performance, caching mechanisms are implemented to store frequently accessed data such as property listings and search results, significantly reducing database load and improving response times. Load balancing techniques are used to distribute incoming traffic evenly across multiple service instances, preventing bottlenecks and ensuring high availability. Database performance is enhanced through optimized queries, indexing, and efficient data access strategies, ensuring quick retrieval and processing of information.

Automated CI/CD pipelines using Jenkins and GitHub Actions help maintain performance standards by continuously testing application updates and preventing performance regressions. Infrastructure provisioning using Terraform or Ansible ensures that the system is deployed on optimized and consistent environments, reducing performance inconsistencies.

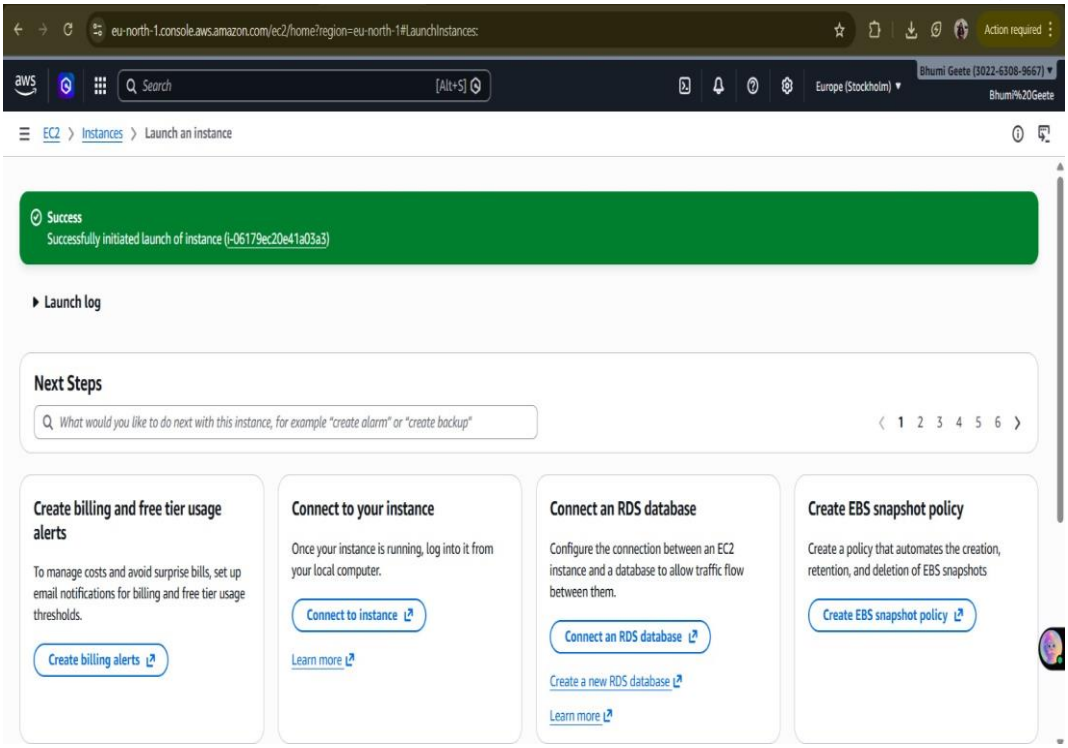
Additionally, monitoring and logging tools are integrated to track system metrics such as response time, CPU usage, and memory utilization, enabling proactive performance tuning and issue resolution. Overall, the system is built to deliver reliable and efficient performance, supporting real-time user interactions and ensuring smooth operation across all environments.

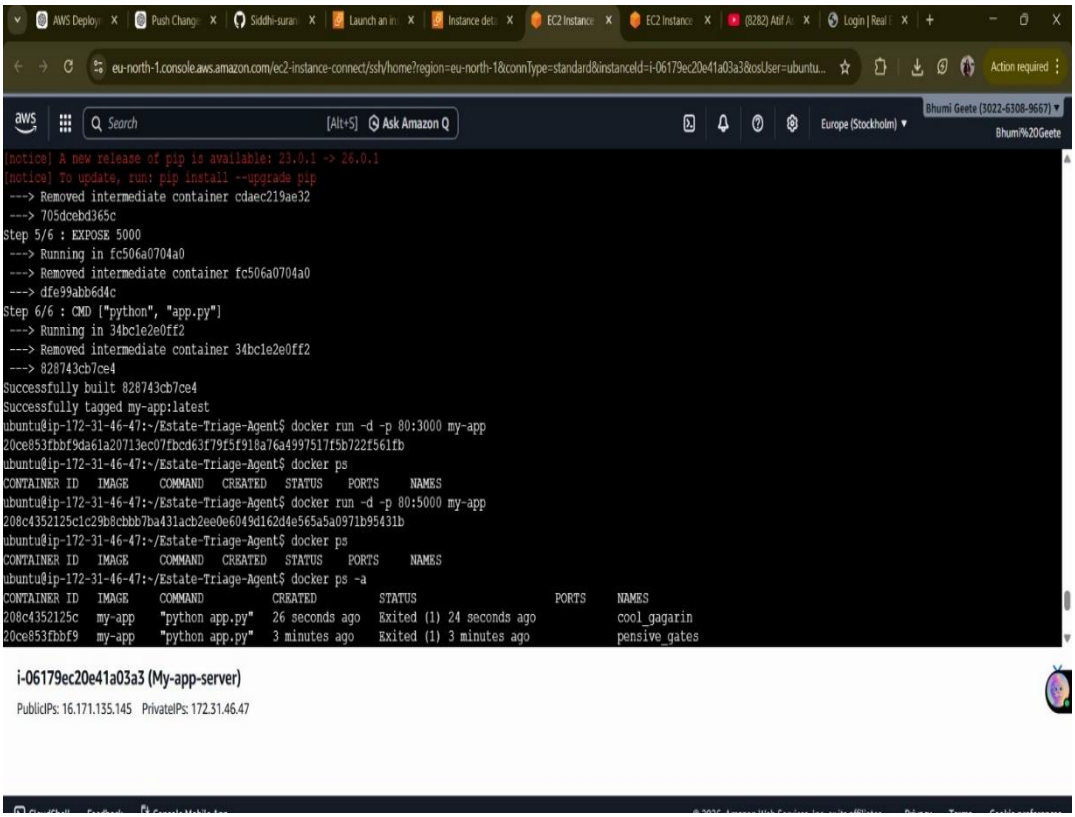
REFERENCES

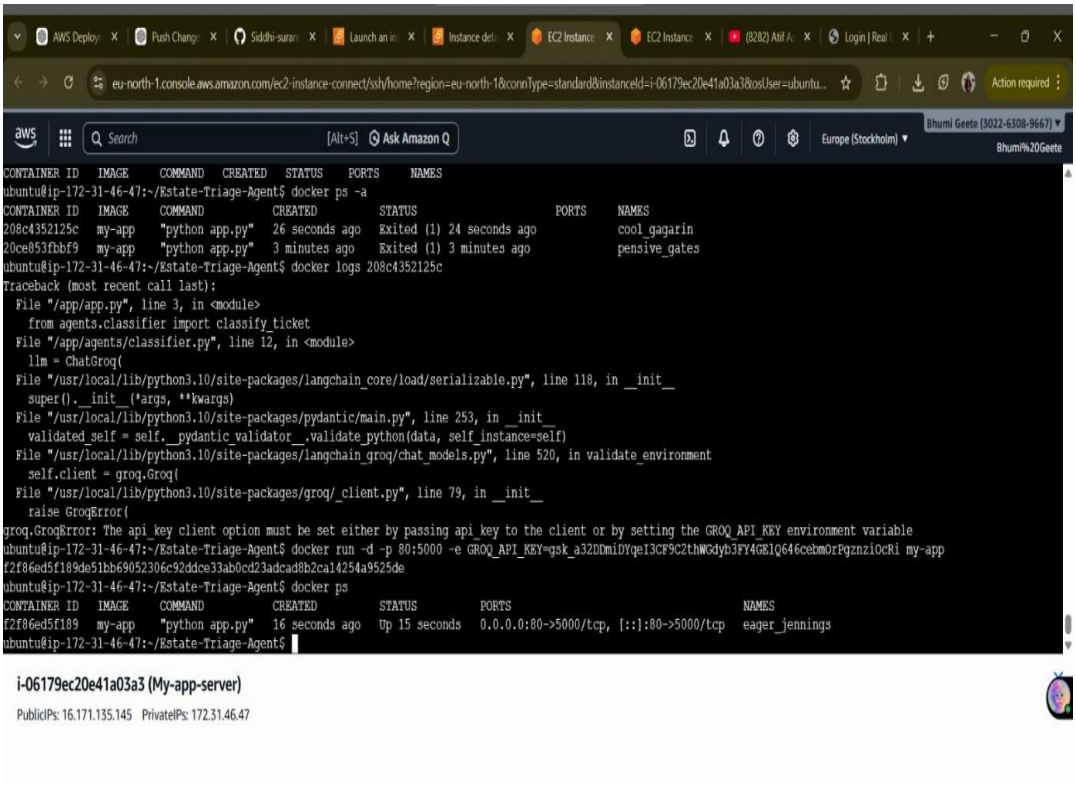
- Official documentation of Terraform for Infrastructure as Code concepts and implementation.
- Official documentation of Ansible for configuration management and automation practices.
- Docker documentation for containerization and environment consistency.
- Kubernetes official guides for container orchestration, scaling, and deployment strategies.
- Jenkins documentation for CI/CD pipeline setup and automation workflows.
- GitHub Actions documentation for workflow automation and continuous integration practices.
- Git official documentation for version control and collaboration.

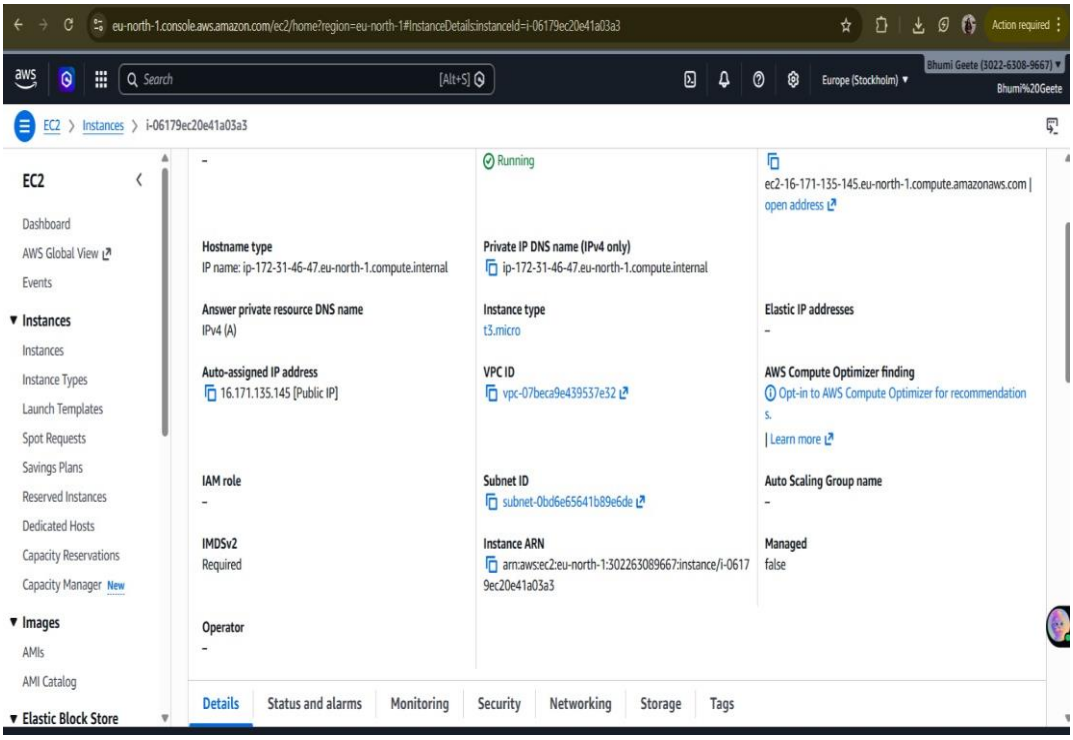
- Amazon Web Services documentation for cloud infrastructure and deployment services.
- Research papers and articles on **DevOps practices and Infrastructure as Code (IaC)** for modern application deployment strategies.
- Industry best practices and guidelines for **microservices architecture, security, and scalable system design**.

```
## Dockerfile


# Starting with a lightweight Python 3.11 image
FROM python:3.11-slim

# Seting /app as the main working directory inside the container
WORKDIR /app

# Copying the requirements file first
COPY requirements.txt .

# Installing all the Python libraries needed for the project
RUN pip install -r requirements.txt

# Copying the rest of the application files into the container
COPY . .

# Exposing port 5000 so the Flask app can be accessed from outside
EXPOSE 5000

# For Starting the Flask application and allow connections from any host
CMD ["python", "app.py", "--host=0.0.0.0"]
```


eu-north-1.console.aws.amazon.com/ec2/home?region=eu-north-1#LaunchInstances:
Action required
Search
[Alt+S]
Europe (Stockholm)
Bhumi Geete (3022-6308-9667)
Bhumi%20Geete
EC2 > Instances > Launch an instance
Success
Successfully initiated launch of instance (i-06179ec20e41a03a3)
Launch log
Next Steps
What would you like to do next with this instance, for example "create alarm" or "create backup"
1 2 3 4 5 6
Create billing and free tier usage alerts
To manage costs and avoid surprise bills, set up email notifications for billing and free tier usage thresholds.
Create billing alerts
Connect to your instance
Once your instance is running, log into it from your local computer.
Connect to instance
Learn more
Connect an RDS database
Configure the connection between an EC2 instance and a database to allow traffic flow between them.
Connect an RDS database
Create a new RDS database
Learn more
Create EBS snapshot policy
Create a policy that automates the creation, retention, and deletion of EBS snapshots
Create EBS snapshot policy


eu-north-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=eu-north-1&connType=standard&instanceId=i-06179ec20e41a03a3&osUser=ubuntu...
Action required
Search
[Alt+S]
Ask Amazon Q
Europe (Stockholm)
Bhumi Geete (3022-6308-9667)
Bhumi%20Geete
[notice] A new release of pip is available: 23.0.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
---> Removed intermediate container cdaec219ae32
---> 705dcebd365c
Step 5/6 : EXPOSE 5000
---> Running in fc506a0704a0
---> Removed intermediate container fc506a0704a0
---> dfe99abb6d4c
Step 6/6 : CMD ["python", "app.py"]
---> Running in 34bc1e2e0ff2
---> Removed intermediate container 34bc1e2e0ff2
---> 828743cb7ce4
Successfully built 828743cb7ce4
Successfully tagged my-app:latest
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker run -d -p 80:3000 my-app
20ce853fbbf9da61a20713ec07fbcd63f79f5f918a76a4997517f5b722f561fb
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker run -d -p 80:5000 my-app
208c4352125c1c29b8cbbb7ba431acb2ee0e6049d162d4e565a5a0971b95431b
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
208c4352125c my-app "python app.py" 26 seconds ago Exited (1) 24 seconds ago cool_gagarin
20ce853fbbf9 my-app "python app.py" 3 minutes ago Exited (1) 3 minutes ago pensive_gates
i-06179ec20e41a03a3 (My-app-server)
PublicIPs: 16.171.135.145 PrivateIPs: 172.31.46.47


eu-north-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=eu-north-1&connType=standard&instanceId=i-06179ec20e41a03a3&osUser=ubuntu...
Search [Alt+S] Ask Amazon Q
Europe (Stockholm)
Bhumi Geete (3022-6308-9667)
Bhumi%20Geete
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
208c4352125c my-app "python app.py" 26 seconds ago Exited (1) 24 seconds ago cool_gagarin
20ce853fbbf9 my-app "python app.py" 3 minutes ago Exited (1) 3 minutes ago pensive_gates
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker logs 208c4352125c
Traceback (most recent call last):
File "/app/app.py", line 3, in <module>
from agents.classifier import classify_ticket
File "/app/agents/classifier.py", line 12, in <module>
llm = ChatGroq(
File "/usr/local/lib/python3.10/site-packages/langchain_core/load/serializable.py", line 118, in __init__
super().__init__(*args, **kwargs)
File "/usr/local/lib/python3.10/site-packages/pydantic/main.py", line 253, in __init__
validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
File "/usr/local/lib/python3.10/site-packages/langchain_groq/chat_models.py", line 520, in validate_environment
self.client = groq.Groq(
File "/usr/local/lib/python3.10/site-packages/groq/_client.py", line 79, in __init__
raise GroqError(
groq.GroqError: The api_key client option must be set either by passing api_key to the client or by setting the GROQ_API_KEY environment variable
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker run -d -p 80:5000 -e GROQ_API_KEY=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 my-app
f2f86ed5f189de51bb69052306c92ddce33ab0cd23adcad8b2ca14254a9525de
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
f2f86ed5f189 my-app "python app.py" 16 seconds ago Up 15 seconds 0.0.0.0:80->5000/tcp, [::]:80->5000/tcp eager_jennings
ubuntu@ip-172-31-46-47:~/Estate-Triage-Agent$
i-06179ec20e41a03a3 (My-app-server)
PublicIPs: 16.171.135.145 PrivateIPs: 172.31.46.47


eu-north-1.console.aws.amazon.com/ec2/home?region=eu-north-1#InstanceDetails:instanceId=i-06179ec20e41a03a3
Search [Alt+S]
Europe (Stockholm)
Bhumi Geete (3022-6308-9667)
Bhumi%20Geete
EC2 > Instances > i-06179ec20e41a03a3
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Running
ec2-16-171-135-145.eu-north-1.compute.amazonaws.com | open address
Hostname type
IP name: ip-172-31-46-47.eu-north-1.compute.internal
Private IP DNS name (IPv4 only)
ip-172-31-46-47.eu-north-1.compute.internal
Answer private resource DNS name
IPv4 (A)
Instance type
t3.micro
Elastic IP addresses
Auto-assigned IP address
16.171.135.145 [Public IP]
VPC ID
vpc-07beca9e439537e32
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. | Learn more
IAM role
Subnet ID
subnet-0bd6e65641b89e6de
Auto Scaling Group name
IMDSv2
Required
Instance ARN
arn:aws:ec2:eu-north-1:302263089667:instance/i-06179ec20e41a03a3
Managed
false
Operator
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags